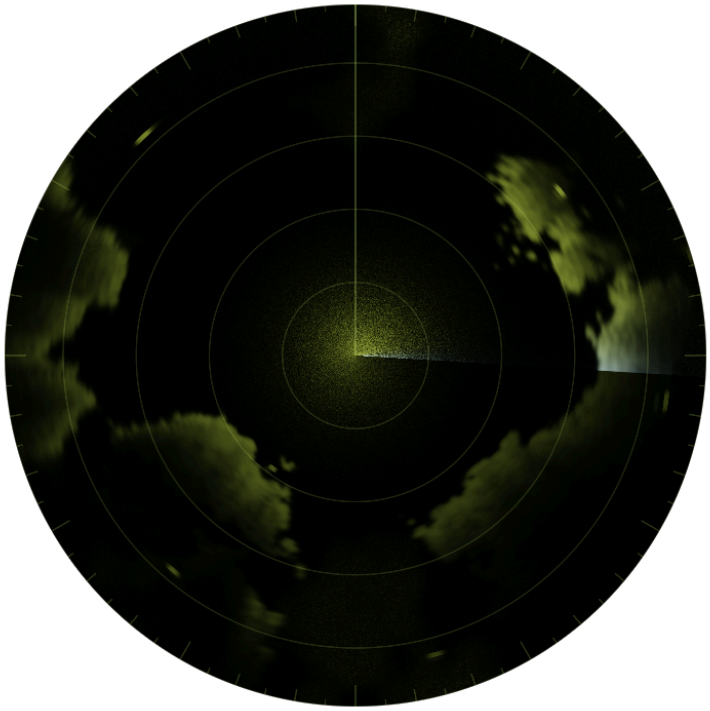
Radar user guide

Radar is **a plan-position radar scope, for Resolume Arena and Avenue**, as two FFGL plugins in one download: **SW Radar**, a source that's a radar watching a synthetic sea, and **SW Radar Over**, an effect that makes your clip the sea.

Rather than drawing a picture of a radar, it puts a reflectivity field through a model of one, and the look falls out of the model: ships painted as arcs as wide as the beam and streaks as long as the pulse, a trail behind the sweep, a bright centre that STC pushes down, clutter, rain, coasts with shadows behind them, and moving contacts that leave the plot of their track.



SW Radar at its defaults after ten seconds, rendered by the offline harness rather than captured from Resolume: a harbour approach on a 24 km scope, a bay open to the north, the sweep just past east with the trail fading behind it.

Before you rely on this: it's released at **v0.2.0**, and it's still early days. The scope is measured rather than asserted, by a harness that drives the real plugin classes headlessly, at two rasters and on a software renderer: a point target's arc has a half-power width of Beamwidth to 0.004 degrees; its streak is c tau / 2 long to 8×10^{-5} of a range bin, running outward; every bearing is painted exactly once a rotation at whole, fractional and jittered frame rates; equal targets fall with range with a slope of −3.9998 against the radar equation's −4, and are flat with STC at n = 4; the phosphor holds its two-term decay to 2×10^{-7}. Thirty-two models that are wrong on purpose are each shown to fail their check, and one-character mutations of the shipped shaders and code are caught. All 71 parameters over both plugins are shown to change the picture. **The checks verify the stated model, not a real radar**: the phosphor's time constants are stretched to video rates on purpose, and the synthetic sea is made to look right, not measured (see Known limits). It's **never been loaded into Resolume on macOS**; the one host it's run in there is the fleet's own test host, `oxbow`, for 120 frames each. **The Sprites group is new in v0.2.0.** It's measured by the same harness (what a file becomes, the threshold, the count, size and orientation off the painted picture, the speed off the phosphor, the beam revealing them and the phosphor letting them go, the same frame twice), runs in the browser demo, and has been in Resolume Arena on Windows (below), never on macOS. A build of the v0.3.0 code (CI's MSVC DLLs from 3943847, the same but for the version number) loads, registers and renders in Resolume Arena 7.27.1 on software rendering (win-lab, Mesa llvmpipe, no GPU, no sound device), with every control matching what the plugins declare (43 and 41 host controls) and Arena's log clean, in the fleet's Arena gate: 17 of 17 checks, with the audio rows skipped, and the test alien loaded as the Sprite File in both. The sweep turns, so no two grabs of the picture are alike, which puts the gate's noise floor high, and it could only confirm some controls one at a time: 11 of the source's 35 valued controls and 8 of the effect's 34, with the rest inconclusive (none dead). The v0.3.0 controls are among the inconclusive ones, as they would be: Path and Start With only act as sprites start. The harness's own sweep (every one of the 71 parameters moves the picture) covers the rest. v0.2.0 passed the same gate with 40 and 38 controls, v0.1.0 with 29 and 27.

The sprites were also tried by hand in SW Radar, ten of them. At v0.2.0: the built-in blip, the test alien as a PNG, the same PNG from a folder named with accents, emoji and katakana, a file that isn't a picture (the blip, and a warning in the plugin's log naming the file and why), the alien as a JPG (read by its brightest channel), then Hue, Saturation and Face Centre. At v0.3.0: Transit with Start With 0 (they trickled in at the top and slid down), the picture's own colour (the alien glowed green), Head In and Face Centre on Transit, then Closing with Start With 3 (three near the rim, the rest arriving) and back to Phosphor. Each one looked as the harness says it should. Software rendering says nothing about a GPU or about speed.

New in v0.3.0: Path, Start With, Head In and Sprite Colour (see the Sprites group). The harness measures them off the painted picture (the Transit track column by column, the count on the scope rotation by rotation, which way up a sprite is, its colour), they run in the browser demo, and they've been in Resolume Arena on Windows (above). At their defaults the plugins draw exactly what v0.2.0 drew.

> Try it on a spare layer before you put it in a show.
>
> This codebase was created with AI assistance, directed and reviewed by a human author.

Installing

Every download has both plugins in it: **SW Radar** (a source) and **SW Radar Over** (an effect). Drop them into Resolume's effects folder and restart Resolume:

```
macOS    ~/Documents/Resolume Arena/Extra Effects/
Windows  %USERPROFILE%\Documents\Resolume Arena\Extra Effects\
```

Avenue uses the same layout under its own folder name. SW Radar turns up among the sources and SW Radar Over in the effects browser.

The macOS download is a universal build (Apple silicon and Intel), as a `.dmg` or a `.zip`. It's Developer ID-signed and notarised by the release pipeline after publication, so the bundles just load. If macOS refuses a download, it predates the signing, so download it again. The Windows download is an x64 installer or a `.zip`. It isn't code-signed, so the installer trips SmartScreen once: **More info** → **Run anyway**.

A scope shows the sea convolved with the radar

A plan-position indicator (PPI) is a cathode-ray tube whose trace runs out from the centre along the antenna's bearing while the antenna turns. The transmitter sends a pulse; everything the pulse hits sends an echo back; the echo from range r arrives $2r/c$ later and brightens the trace at that radius. A long-persistence phosphor holds each paint until the antenna comes round again. So what the scope shows is the sea smeared by the radar that looked at it, and each smear is something you can see.

the radar	what comes out
the beam has a width	**a point target paints an arc** as wide as the beam: a far ship is a long arc, a near one a short one, and a coast is smeared round in bearing
the pulse has a length	**every echo is a radial streak** c tau / 2 long (150 m per microsecond of pulse), running outward from the target's range
the antenna turns	**the sweep leaves a trail**: what was painted a moment ago is bright, what was painted a rotation ago is about to be repainted
the radar equation	**near returns are huge**: a point target's echo falls as the fourth power of range, so without correction the centre blooms and the edge is dark
sensitivity time control (STC)	**the gain rises with range** as R to the power n; at n = 4 the radar equation is exactly undone and equal targets look equal everywhere
sea clutter, rain, noise	**clutter crowds the centre**, rain comes as speckled cells, noise as speckle everywhere, each different from pulse to pulse
a coast facing the radar	**a bright edge with shadow behind**: the land nearest returns hard and hides the land behind it
moving targets	**the plot of a track**: each paint of a moving contact is left fading where it was

Nothing is painted as a line per frame. Every frame paints exactly the bearings the antenna swept past since the last one, the swept wedge, into a fixed polar grid of 2048 bearings by 1024 ranges, so every bearing is painted once a rotation whatever the frame rate, and resizing the output touches none of it.

Start here

1. Put **SW Radar** in a clip slot and trigger it. The scope starts black and fills in over the first rotation, 2.5 s at the default 24 rpm: a bay open to the north on a 24 km scope, a few contacts, clutter round the centre.
2. Turn **Beamwidth** up. Every contact becomes a wide arc, and the coast smears round.
3. Turn **Pulse Length** up. Every contact grows a radial tail.
4. Turn **STC** all the way down to see the raw radar equation (the centre blooms), then all the way up (n = 4: flat).
5. Change **Range**: the coast zooms, the contacts stay (see *Contacts live on the scope* below).
6. For your own footage: put **SW Radar Over** on a clip. The clip's bright parts become echoes. Bring **Mix** down to see the clip under its own echoes.
7. For contacts of your own: raise **Sprite Count**, then choose a PNG in **Sprite File** (see *The Sprites group* below). Without a file they're radar blips.

The Antenna group

Resolume shows every slider as 0 to 1; the ranges below are what the ends of each slider mean.

RPM (0; 1 to 120 rpm, default 24). How fast the antenna turns. At 0 the antenna stops and paints nothing more; the picture fades by its own persistence. Faster reads as nervous, slower as a long-range search.

Beamwidth (0.5 to 20 degrees, default 2). The width of the arc a point target paints, measured at half power. This is the **two-way** width, the one you see on the scope. An antenna datasheet quotes the one-way width, which for the same antenna is about 1.4 times (the square root of 2) wider: set 1.4 degrees here to see a "2-degree" antenna. *I chose this so the number on the control is the width on the screen.* A wider beam blurs a coast but doesn't brighten it (see Known limits).

Sidelobes (0 to 1, default 0.35). The shape of the beam: 0 is a Gaussian, 1 is a uniform aperture's pattern, whose sidelobes paint faint extra arcs either side of a strong target. Both have the same half-power width, so Beamwidth means the same at any setting.

Direction (*Clockwise*, *Anticlockwise*). Which way the antenna turns.

The Transmitter group

Pulse Length (0.05 to 20 µs, default 2). The streak every echo leaves, c tau / 2 long: 150 m per microsecond, so 2 µs is 300 m. On a 24 km scope that's a short tail; at 12 µs it's 1.8 km and every contact becomes a radial block.

Range (0.5 to 200 km, default 24). The scope's radius. The rings are spaced at a round number of km, four to seven of them.

Gain (−30 to +50 dB; default 0 in the source, +8 in the Over). The receiver's gain.

STC (0 to 4, default 2.6 in the source, 4 in the Over). The exponent n of the gain's rise with range. At 0 you see the radar equation raw: returns fall as the fourth power of range, the centre blooms and far targets vanish. At 4 the gain rises as fast as the returns fall and equal targets look equal at any range. Noise enters before the gain, so STC pushes the noise down near the centre too. The Over defaults to 4 so the far side of your clip isn't four orders of magnitude dimmer than the near.

The Returns group

Clutter (0 to 1; default 0.4 in the source, 0.2 in the Over). Sea clutter: waves returning the pulse near the ship, strongest at the centre and falling off within a few km, speckled pulse to pulse. It's an area return, so it falls with range as R^{-3} rather than a point target's R^{-4}.

Noise (0 to 1, default 0.3). Receiver noise: speckle everywhere, before the STC gain.

The source only:

Contacts (0 to 16, default 7). Moving targets on straight tracks, respawned when they leave the scope. Each one leaves the plot of its track in its fading paints.

Rain (0 to 1, default 0.4). Rain cells: volume returns, speckled pulse to pulse, drifting with the wind. At the defaults the rain is faint behind the land and the gain; over open water (Land 0) with 10 dB more Gain it shows as speckled blobs.

Land (0 to 1, default 0.7). How much of the synthetic map is land. The coast facing the radar returns hard; the land behind it falls into shadow (1.5 km of land hides what's behind it).

Seed (0 to 9999, default 11). Which synthetic coast. The default is a bay open to the north. The map is in km, so Range zooms one map; the land is kept away from the ship itself so no seed puts the radar inside a continent.

The Over only:

Threshold (0 to 1, default 0.2). How bright a part of the clip must be to echo. The clip's **brightest channel** is used, not its luma, so a saturated blue ring echoes as well as a white one. The clip's alpha multiplies it.

The Audio group

Audio (Resolume's FFT input). **Audio Strobe** (0 to 1, default 0): each onset in the sound is a burst of interference along the next part of the sweep, a radial spoke, the way a jammer or another radar shows on a real scope. **Audio Contacts** (the source only, 0 to 1, default 0): each onset puts a strong echo 20 to 60 degrees ahead of the sweep, which the sweep then finds and which lives three rotations. Both are amounts; 0 is off. The first frame after a clip trigger fires nothing, even if the music is already loud.

A regular beat gives spokes that **stand still**: a beat every half second at 24 rpm is a fifth of a turn, so five spokes come back at the same five bearings every rotation. Change RPM, or play something less regular, to make them walk.

The Scope group

Persistence (0.1 to 30 s, default 1.6). The afterglow's time constant. At the default the trail visibly fades within the 2.5 s rotation; at 8 s several rotations linger at once.

Flash (0 to 4, default 1.5). The strength of the phosphor's fast flash at the sweep's leading edge, against the afterglow's 1.

Phosphor (*P7*, *P19*, *Green*). *P7* is the classic radar tube: a blue-white flash and a yellow-green afterglow. *P19* is orange all through. *Green* is a single green, like a general-purpose tube. Changing phosphor under a long Persistence mixes the old paints with the new until they're repainted.

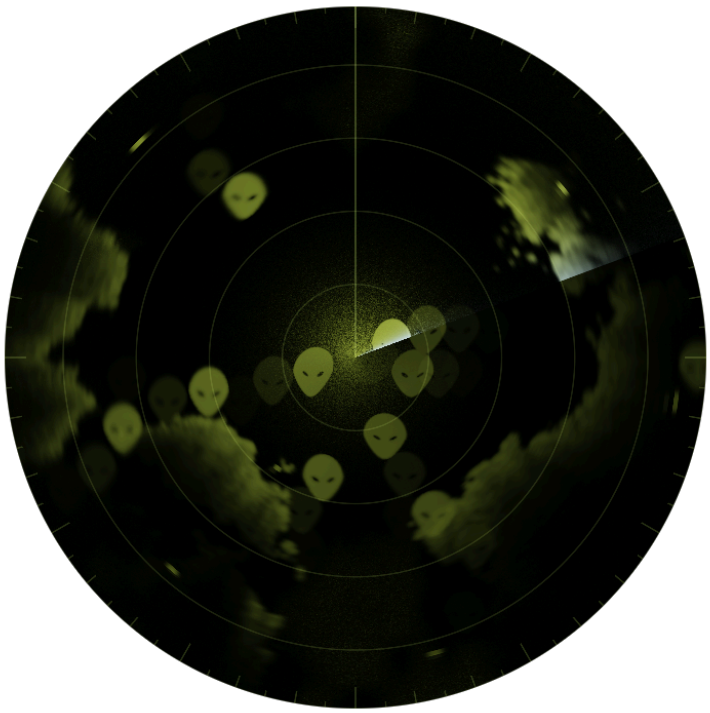
Rings (0 to 1, default 0.35), **Bearing Marks** (0 to 1, default 0.5), **Heading Line** (on in the source, off in the Over). The range rings, the bearing scale round the edge, and the line at the ship's heading (north). Each is one pixel wide.

Scope Size (0 to 1; default 0 in the source, 1 in the Over). From the scope inscribed in the frame (0) to a circle covering the whole frame (1). At 1 the bearing scale is outside the frame.

Mix (the Over only, 0 to 1, default 1). The clip under the scope. At 1 the output is opaque whatever the clip's alpha; at 0 the clip is returned exactly, alpha and all.

The Sprites group: your own contacts

New in v0.2.0, from a request on GitHub (radar#5): a picture of your own as the contacts, coming in from the edge of the scope to the centre. Both plugins have it.



SW Radar at its defaults with Sprite Count 10, Sprite Size 0.14 and the alien from the repository's `tools/fixtures/`, rendered by the offline harness rather than captured from Resolume.

The sprites aren't drawn on top of the scope. Each one is a target the radar sees: the sweep paints it as it passes, the beam smears it sideways and the pulse outward, the same as the coast, and it fades on the phosphor until the sweep comes round again. Between paints it moves on, so every sprite leaves a trail of fading copies of itself, the way a ship leaves the plot of its track. Like the coast, a sprite near the centre is brighter when STC is low (the radar equation), and the Gain moves it with everything else. Sprites don't hide behind land and land doesn't hide behind them: they're contacts, like the source's ships.

They're **off by default** (Sprite Count 0), so a composition saved with v0.1.0 looks the same.

Sprite File (a PNG or JPG; empty is the built-in blip). The picture. With no file, or one that can't be read (missing, not a picture, more than 8192 pixels on a side), you get a radar blip, and the reason goes in the log (see *If it looks wrong*). The file is read once, when you choose it: if you change the file on disk, choose it again.

Sprite Threshold (0 to 1, default 0.5). The picture's alpha over this is the target. A picture with no transparency (a JPG, or a PNG with none) is read by its **brightest channel** instead, as the Over reads its clip: light shapes on dark. On the built-in blip, Threshold sets its size: a disc Sprite Size × (1 − Threshold) across.

Sprite Count (0 to 32, default 0). How many at once. When one reaches the centre it's gone, and a new one starts at the rim on a new bearing (on Transit, when one leaves at the bottom a new one comes in at the top). When you turn them on they start spread out along the way in, so the scope fills at once rather than with a ring at the edge, unless Start With says otherwise.

Sprite Size (0.02 to 0.5 of the scope's radius, default 0.1: about 50 pixels on a 1080p scope). The picture's longer side. Like the source's contacts it's in scope units, so Range zooms the coast and leaves the sprites alone.

Sprite Speed (0, then 0.004 to 0.5 of the radius a second; default 0.04, which is 25 seconds from the rim to the centre, and up to 50 seconds across the scope on Transit). 0 holds them still.

Speed Spread (0 to 1, default 0.35). At 0 they all come in at Sprite Speed. At 1 each one has its own, from a third of it to three times it, drawn afresh each time one starts at the rim.

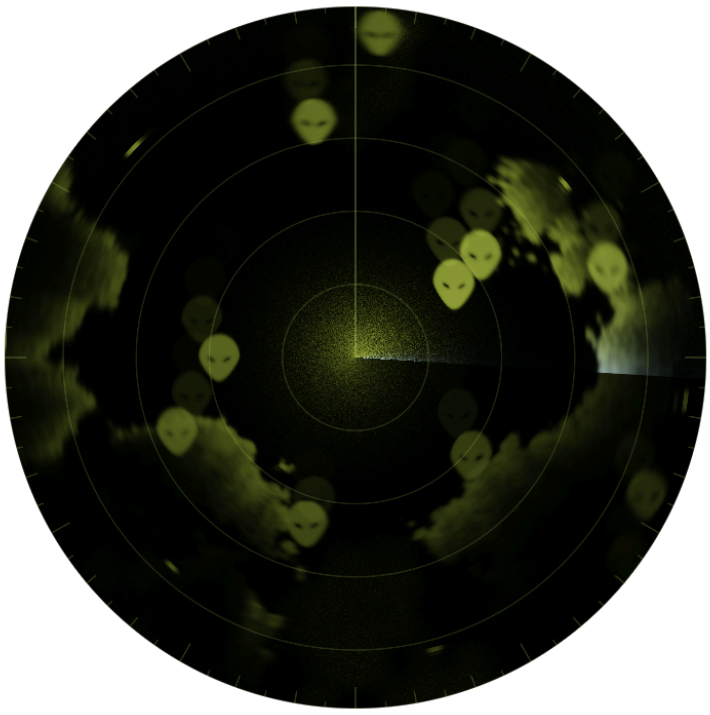
Stability (0 to 1, default 0.75). At 1 each one comes straight in at a steady speed. Lower, and its speed surges and sags (between 0.1 and 1.9 times its own at 0) and it weaves from side to side, up to 20 degrees of bearing. On Transit the weave is across the track, up to 0.15 of the radius, dying away towards the rim so a sprite never wanders off the scope. Speed, Speed Spread and Stability change how fast the sprites are going, never where they are, so moving a slider never makes them jump. The weave is the one part of Stability that would: each sprite keeps the weave it started with, and the next one takes the new setting. (In v0.2.0 a sudden change of Stability threw every sprite round in bearing in one frame.)

Orientation (*Upright*, *Face Centre*, *Head In*). *Upright*: every sprite the right way up on the screen. *Face Centre*: its bottom towards the centre of the scope, like figures standing on a globe. *Head In* (new in v0.3.0): the other way up, head first towards the centre. On Transit both Face Centre and Head In keep facing the centre, so a sprite turns as it goes past, quickly if it passes close.

Hue (0 to 1, default 0) and **Saturation** (0 to 1, default 0.5). The sprites glow in the phosphor's colours, so on P7 they flash blue-white and fade to yellow-green like everything else. Hue turns both colours round the colour wheel (0.33 is a third of the way round); Saturation scales how coloured they are: 0 is grey, 0.5 the phosphor's own, 1 twice as strong. At Hue 0 and Saturation 0.5 a sprite is exactly the colour of every other echo. *I chose to start from the phosphor rather than from a colour you pick, so the sprites still look like they're on the same tube.* With Sprite Colour on *Picture*, Hue and Saturation turn and scale the picture's colour instead.

Grow On Approach (0 to 1, default 0). At 0 they're the same size all the way in. At 1 a sprite is four times Sprite Size by the time it reaches the centre, growing steadily on the way. It's a matter of how far from the centre a sprite is, Sprite Size × (1 + 3 × Grow × (1 – range)), so on Transit a sprite grows until its closest approach to the centre and shrinks again on the way out. One that passes right through the middle reaches four times its size; one that crosses near the edge barely grows at all.

Path (*Closing*, *Transit*; new in v0.3.0). *Closing* is what v0.2.0 did: in from anywhere round the rim, straight to the centre. *Transit* is Steve's idea: the radar is on a ship heading north through things that aren't moving, so each sprite comes into range at the top of the scope, slides straight down past you and leaves at the bottom, and then a new one comes in at the top. Where each one crosses is picked at random across the middle 85% of the scope's width: evenly across, which is how things scattered over the sea would pass a ship, but not hugging the rim, where a track would be a brief scrap of a sprite half off the scope. Only the sprites move. The coast, the rain, the clutter and the source's own contacts stay where they are, so strictly it's a radar watching a convoy steam south rather than one going anywhere. With Land at 0 nobody can tell. Changing Path doesn't move anything already on its way: each sprite finishes on the path it started on, and the next one takes the new path. At the default speed that's a while, so set it before you raise Sprite Count, or turn the speed up for a moment.



SW Radar at its defaults with Sprite Count 10, Sprite Size 0.14, the alien and Path set to Transit, fifteen seconds in: each one is sliding south with its old paints stacked above it. Rendered by the offline harness.

Start With (0 to 32, default 32; new in v0.3.0). How many sprites are on the scope when they start, or when you raise Sprite Count. The rest arrive at the rim (the top, on Transit) one at a time, at irregular moments, and every sprite is on the scope before the first one leaves. The latecomers

arrive within 85% of the quickest crossing anyone could make: the shortest track (one radius closing, about half the diameter on Transit) at the fastest a sprite can go, which is Sprite Speed × 3^Speed Spread with Stability's surge on top. At the defaults that's about twelve seconds, and the last one turns up a couple of seconds before anyone could possibly get to the other side. So that the ones on at the start can't beat them to it, they start near the rim: in the outer 15% of the radius closing, or the first part of their track on Transit. It doesn't matter whether you raise Sprite Count in one go or drag the slider up a step at a time: it's one build-up, with the latecomers spread over whatever is left of the same window.

The default, 32, means all of them at once, spread right along the way in, which is what v0.2.0 did. If Sprite Count goes up while sprites are already on, only enough appear at once to make up Start With, and the rest arrive the same way; the ones already on carry on as they were. At Sprite Speed 0 the latecomers wait with everything else. It only acts when sprites start, so moving it on a running layer does nothing until Sprite Count goes up.

Sprite Colour (*Phosphor*, *Picture*; new in v0.3.0). *Phosphor* is the phosphor's colours, as above. *Picture* takes the average colour of your picture (the parts that count, weighted by how much they count) and glows in that, as bright as the phosphor's colours would be. The alien comes out green. The built-in blip has no colour, and a picture whose average is grey (the black-and-white panda, say) has none to give, so both stay Phosphor. It's one colour per picture, not the picture's colours pixel by pixel: a radar still can't tell what colour anything is.

Making a good sprite

- **A PNG with transparency.** The radar sees a silhouette: the shape where the alpha is over Sprite Threshold. The colours inside the shape are ignored; Hue and Saturation colour it instead (or, with Sprite Colour on *Picture*, its average colour does).
- **Pulse Length all the way down keeps the picture whole.** Steve's tip from radar#5: the pulse smears every echo outward, and at the default 2 µs a detailed sprite turns into a smudge. Take Pulse Length to the bottom and it's recognisable again. Lowering Beamwidth does the same for the sideways smear.
- **Holes count.** Make eyes, mouths and gaps transparent if you want them to show. The alien in the repository's `tools/fixtures/` has its eyes cut out; the panda's eye patches are opaque, so it's a silhouette with ears.
- **Bold and simple.** The beam smears every sprite sideways by Beamwidth and outward by the pulse, so thin lines and small details vanish, more so towards the edge where the beam is wider. A chunky silhouette survives; a detailed illustration becomes a blob. Lower Beamwidth and Pulse Length to sharpen them.
- **Small is fine.** The plugin reduces any picture to 128 pixels on its longer side, so a 128 × 128 PNG is plenty.
- **No transparency? Light on dark.** A JPG is read by its brightest channel, so a white skull on black works and a black skull on white gives you a white square with a skull-shaped hole. Raise Sprite Threshold to drop a grey background.

- **Emoji:** save it as a PNG with a transparent background (most emoji sites and design tools will do that). A screenshot of one comes with an opaque background and reads as a square.
- **Square pictures read best.** A wide picture is drawn wide; Sprite Size is its longer side.

SW Radar Over: your clip is the sea

The clip is laid under the scope in range and bearing, and its brightest channel above Threshold, times its alpha, is the reflectivity. The radar then does what it does to anything: bright shapes become echoes smeared by the beam and the pulse, painted by the sweep and fading behind it. The echoes are drawn in the **phosphor's** colour, not the clip's: a radar sees reflectivity, not colour. *That's on purpose.* Bring Mix down to put the clip's own colour back under the echoes.

Point it at something with bright, separate shapes: Resolume's Trinity (rings), BattleWeapon Tank and the SpaceUniverse astronaut all read well. A clip that's bright everywhere becomes a solid disc; raise Threshold.

Contacts live on the scope

The coast, the rain and the clutter are in km: Range zooms them. The source's **contacts are in scope units**: they keep their size and their speed on the screen whatever the Range. A ship at 20 knots on a 24 km scope moves a pixel every few seconds, which doesn't read as traffic, and a real harbour doesn't fill up with faster ships when you zoom out. *I chose that for the picture's sake.*

How it works

- **The antenna** turns at RPM × the host's real elapsed time (the clock is Resolume's, in double precision: Resolume's clock runs to hundreds of millions of milliseconds, where a float can't resolve a frame). Each frame paints the bearing bins whose centres it crossed, and consecutive frames' intervals abut, so each bin is painted exactly once a rotation.
- **The echo** is, per range bin: the surface (the synthetic map, or the clip) averaged over the pulse's footprint and convolved with the beam, plus clutter, rain and point targets, times the radar equation's R^{-4} and the STC gain, plus noise; the video is 1 − e^−P.
- **Point targets are analytic**: sigma times the beam at their bearing offset times the fraction of the range bin their pulse covers.
- **The beam** is a two-way power pattern, 1 on the axis and exactly ½ at ± Beamwidth/2.
- **The phosphor** is two components per texel, a flash and an afterglow. Each texel stores its value at its bearing's last paint, and the decay since then is computed exactly on the CPU for every bearing and applied when the scope is drawn, so it is exact at any frame count.

- **Sprites** are reflectivity, like the coast: each frame the CPU moves them (in double precision, from the host's clock, so the same clock gives the same frame, on either path, and a latecomer arrives at the same moment at any frame rate), and the pass that resamples the surface into range and bearing samples each sprite's picture there too, averaged over the pulse; the paint convolves it with the beam. The phosphor keeps the sprites' share of every echo beside the whole, decaying the same way, which is what lets Hue and Saturation colour them alone.
- **No sin, cos or atan** in the shaders: GLSL doesn't bound their error, and a software renderer's sine made a beam 3% too wide. The shaders use their own, built from bounded operations.

Performance

At the defaults, on an Apple M4 Max shared with other builds, the median frame is **0.34 ms at 720p, 0.39 at 1080p and 0.74 at 4K** for SW Radar, and **0.28, 0.43 and 1.17 ms** for SW Radar Over: a few percent of a 60 fps frame. The GPU paints only the wedge the antenna swept; the grid is fixed at 2048 × 1024 whatever the output size. Sixteen sprites cost nothing the same benchmark could measure on that GPU (each one is only worked on where the beam can reach it). On a software renderer they're dearer: about an eighth of a second a frame more for twelve big ones at 320 × 180 on Apple's.

Since the sprites the phosphor has four 32-bit channels per texel instead of two (the sprites' share sits beside the whole), so each instance holds more GPU memory for its buffers: about 88 MB for SW Radar and 80 MB for SW Radar Over, where v0.1.0 held 48 and 40.

If it looks wrong

The scope is black. It fills in over the first rotation. At RPM 0 nothing is painted. In the Over, a dark clip under Threshold returns nothing: lower Threshold or raise Gain.

The Over is a solid disc. The clip is bright everywhere: raise Threshold.

The centre is a bright blob. STC is low. Raise it.

The far edge is empty. STC is low or Range is long: at 150 km only coastlines show, because land hides what is behind it and the sea returns little.

I can't see the rain. Lower Land, raise Gain, and raise Rain.

The spokes stand still. A regular beat at a rotation that divides it (see the Audio group).

No sprites. Sprite Count is 0 by default; raise it.

My sprite is a radar blip. No file is chosen, or the file couldn't be read: the log says which file and why ("cannot be opened", "will not decode"). Choose it again once it's fixed.

My sprite is a square. The picture has no transparency, and its background is bright: it's read by its brightest channel. Save it with a transparent background, or put it light on black.

My sprite is a blob. It's too detailed for the beam: see *Making a good sprite*, or lower Beamwidth and Pulse Length.

I changed Path and nothing happened. Sprites already on their way finish on the path they started on. Wait for them, or raise Sprite Speed for a moment, or set Sprite Count to 0 and back.

Start With does nothing. It only acts when sprites start: set Sprite Count to 0, choose Start With, then raise Sprite Count.

Sprite Colour Picture looks like Phosphor. The picture's average is grey, or it's the built-in blip. Neither has a colour to give.

Old echoes linger after a change. The phosphor keeps what it was painted until the sweep comes round, and with a long Persistence for longer.

Neither plugin is in the browser. Check the folder under Installing, and that Resolume was restarted.

It does nothing at all. A shader that won't compile looks just like that, and the actual error is in the log:

```
macOS    ~/Library/Logs/radar/radar.YYYY-MM-DD.log
Windows  %LOCALAPPDATA%\radar\logs\radar.YYYY-MM-DD.log
```

It records the GL vendor, renderer and version at load, and which shader failed if one did.

Known limits

- **The phosphor is stretched to video rates, and simplified.** A real P7 is a cascade screen: a silver-activated zinc sulphide layer gives the blue-white flash (peak near 440 nm) and a copper-activated zinc-cadmium sulphide layer the long yellow afterglow (near 558 nm), lasting over a minute in low light. The fleet's source for this is Patrick Jankowiak's compilation of the EIA/JEDEC phosphor tables (*Cathode Ray Tube Phosphors Of Interest To The Experimenter*, 2010), checked for this release; the JEDEC standard itself (TEP116-C) wasn't consulted. Here the flash lasts a few frames (35 ms for P7, 25 ms for P19, 60 ms for Green) so a 60 fps picture can show it, and the afterglow is an exponential with a time constant you set. The same tables list P7's long component as an inverse power law, not an exponential, so a real tube's trail has a longer, fainter tail than this one. P19's orange is (KF,MgF_2):Mn in the same tables. The colours are chosen to look like the tubes, not computed from spectra.

- **Distributed returns are normalised.** Physically the return from land, sea clutter and rain grows with the size of the resolution cell, so a wider beam or a longer pulse brightens them as well as blurring them. Here both only blur, so that Beamwidth and Pulse Length aren't also gain controls. Point targets follow the radar equation exactly. *I chose that for the operator's sake.*
- **Beamwidth is the two-way width** (see the Antenna group).
- **The synthetic sea is made to look right**, not measured: the fBm coast, the 1.5 km shadow rule, the clutter law, the rain, and the contacts' speeds.
- **The radar equation, STC and c tau / 2 are textbook**: a point target's return falls as R^{-4}, STC undoes it with a gain rising with range, and a pulse tau long occupies c tau / 2 in range. They were checked against published summaries for this release (Wikipedia's article on sensitivity time control, Cambridge Pixel's note on STC); the textbook itself (Skolnik) wasn't re-read. Clutter's R^{-3} and rain's R^{-2} follow from the resolution cell's area and volume.
- **No sector scan**: Direction is clockwise or anticlockwise, never back and forth over an arc.
- **Ships and aircraft share one speed range**, in scope units.
- **Sprites are silhouettes.** The radar sees a shape, not a picture: the colours of your file are thrown away (Sprite Colour's *Picture* keeps their average and nothing else), and every sprite is coloured alike. All sprites share one picture.
- **Two paths and no more.** Closing (rim to centre) and Transit (top to bottom); no sectors, no other headings, no paths of your own.
- **The sprites have been in Resolume only on Windows**, in Arena on software rendering (see the top of this guide), never on macOS; the file parameter's dialog there is untested in a host.
- **Never loaded into Resolume on macOS.** Everything numeric was compiled, rendered and measured offline against the real plugin classes in a headless GL context, plus an `oxbow` load. No real audio has reached it in a host: the audio controls were checked with a synthetic spectrum only.
- **Never seen on camera footage**, only on Resolume's bundled CG loops and the harness's card.
- **Only ever run on an Apple M4 Max**, although the macOS build contains an Intel slice. On Windows, see the note at the top of this guide.
- **No presets**, no OpenFX version.
- **There's a browser demo** at radar-demo.stoatworks-labs.com. It's a port to a web page rather than the plugin: the shaders run unedited in WebGL2, and the sweep's crossing rule, the per-bearing timing, the clock and the sea's contacts and rain are rewritten in JavaScript. It has no audio, so the audio controls are left off, and the page lists what else it doesn't reproduce.

About

The last group, **About**, has the plugins' name, version, licence and maker, and buttons that open this user guide (stoatworks-labs.com/software/radar/guide/), the project page, the source on GitHub and the support page in your browser.

Reporting something

github.com/stoatworks-labs/radar/issues. Shout there if something's wrong. A screenshot, which plugin, its Antenna and Transmitter settings, and the composition's resolution and frame rate are usually enough for me to go on. If it did nothing, attach the log.